4. Current architecture

```
Layer | Current component | Role
Managed build API | Compiler/WasmBridge.csproj -> WasmBridge.Core.dll | .NET 4.0 API for compilation,
packaging, validation and external tool invocation.
Managed CLI | CLI/WasmBridge.CLI.csproj -> WasmBridge.exe | Thin command-line frontend for
build/package/verify/self-test and optional host toolchain commands.
Native XP sample | Native/WasmBridge.Native.vcxproj -> WasmBridge.Native.dll | v141_xp DLL containing portable
C algorithms and allocator tests; Release CRT is statically linked.
Portable C core | Core/math.*, Core/image.*, Core/buffers.* | Algorithms compiled both to native XP and
wasm32.
Browser loader | Runtime/wasmbridge.js | Loads Wasm bytes, instantiates, validates expected exports, selects
JS fallback on failure.
Memory adapter | Runtime/buffers.js | Owned handles, allocation/release, typed copies, memory growth, bounds
checking.
Legacy helper | Runtime/legacy-compat.js | Optional capability report and XHR ArrayBuffer loader if fetch is
absent.
Examples | Examples/HelloWorld, ImageProcessing, BufferArena | Manual and regression-facing browser
demonstrations.
Tests | Tests/* | Node host regressions and portable/native C checks; XP Firefox remains final acceptance
target.
```

Output layout: bin\<Configuration>\WXP\x86\ and bin\<Configuration>\WXP\x64\

5. Behavior already verified on actual Windows XP / Firefox 52

These are the strongest known acceptance results. They were observed in the user's real Firefox 52 environment on Windows XP x64 and should be treated as regression baselines.

```
Milestone | Backend(s) | Observed result | Meaning
HelloWorld add() | Wasm + forced JS fallback | PASS: add(20,22) = 42 | A non-empty exported function executes
in FF52/XP; fallback path also works.
RGBA 256 x 160 | Wasm + forced JS fallback | PASS; bytewise reference agreement; input unchanged | Linear
memory transfer, Canvas ImageData, alpha preservation, and fallback parity work.
BufferArena 256 x 256 | Wasm with JS comparison | PASS; independent buffers, released-handle rejection, active
allocations 0 | Dynamic allocator/adapter behavior works in the target browser.
BufferArena memory growth | Wasm | Linear memory 131,072 -> 4,456,448 bytes | WebAssembly.Memory.grow plus
re-acquired views work in Firefox 52.
BufferArena indicative timing | Wasm vs JS | 0.67 ms vs 2.00 ms mean end-to-end; startup/validation 92 ms |
Useful smoke benchmark only; not a general performance claim.

Important: The BufferArena screenshot proves the 256 x 256 path. Other listed image sizes remain useful
regressions, but exhaustive resolution testing is not required before API work. Continue running
representative large/edge cases while the API evolves.
```

6. Native DLL issue at handoff

The old runner log reports that the managed CLI starts and the header parser passes, but P/Invoke then fails with error 0x8007007E while loading WasmBridge.Native.dll.

```
PASS: valid and invalid headers handled.
ERROR: Unable to load DLL 'WasmBridge.Native.dll': The specified module could not be found. (0x8007007E)
DBGRun result: EXIT code=1
```

Subsequent manual inspection narrowed the problem considerably:

* The DLL exists in the locally inspected build output under E:\GPT\CODEX\CODEX\GIT\WasmBridge\bin\Release\WXP\x64\WasmBridge.Native.dll.
* dumpbin /dependents on the built DLL reports only KERNEL32.dll. The native Release project is configured with static CRT linkage.
* The legacy AI_RUN script/log used an R:\GPT\... worktree/path, while the DLL was manually verified under E:\GPT\.... This strongly suggests a stale/mismatched worktree or deployment path rather than a missing MSVC runtime.
* The new CODEX workflow should eliminate this class of error by building and running from one controlled worktree and executing the self-test directly from the output directory.

Recommended direct native check (no AI_RUN/AIEXE)

```
cd /d <repo>\bin\Release\WXP\x64
WasmBridge.exe self-test
```

Expected success markers from the current CLI:

* PASS: valid and invalid headers handled.
* PASS: native RGBA8 inversion, alpha and bounds handled.
* PASS: XP native independent buffers, RGBA output, release and bounds.
* A native 512 x 512 RGBA benchmark line, followed by process exit code 0.

7. Firefox 52.9 ESR compatibility contract

* Classic scripts only for the legacy runtime: no type="module", top-level import/export, dynamic import, async/await, optional chaining, BigInt, or other post-FF52 syntax.
* Use ordinary functions, var declarations, Promise chains, ArrayBuffer, and TypedArrays in handwritten runtime code.
* Runtime/legacy-compat.js is optional. It reports capabilities and supplies XHR ArrayBuffer loading when fetch is absent. It does not patch working globals or polyfill Promise/ArrayBuffer.
* If native WebAssembly is missing or a module cannot instantiate, use a tested JS backend. Do not pretend unsupported Wasm proposals can be polyfilled by syntax shims.
* Validate each module's exports and memory contract after instantiation. Feature presence alone is insufficient.
* The user's XP installation has successfully loaded current samples from file://. Do not assume all deployments will; same-origin HTTP remains a fallback.
* Threading, SIMD, reference types, bulk-memory extensions, memory64, multi-memory, and other newer Wasm features must not enter the legacy profile unless intentionally supported and tested.

8. Existing-library integration already added

The codebase now has optional wrappers for build-host tools so WasmBridge does not need to reinvent compilers, optimizers, validators, or automatic JS translation. These tools are external build dependencies, not XP runtime dependencies.

```
Tool | Current use | Current status
WABT / wasm-validate | validate-legacy command; rejects non-MVP proposal families | Wrapper implemented; not yet run with the actual external executable in the user build environment.
Binaryen / wasm-opt | optimize-legacy command using MVP feature profile | Wrapper implemented; optimized output is revalidated.
Binaryen / wasm2js | generate-fallback command; Wasm -> JavaScript candidate | Wrapper implemented; candidate still experimental.
esbuild standalone | Bundles wasm2js ES module to classic IIFE targeted at firefox52 | Wrapper implemented; use build-host executable, not XP runtime.
Emscripten | Future richer C/C++ compilation backend | Investigated conceptually; not yet integrated into this repository.
```

Existing CLI commands:

```
WasmBridge.exe validate-legacy --wasm module.wasm --validator <path-to-wasm-validate.exe>
WasmBridge.exe optimize-legacy --wasm module.wasm --out optimized.wasm --validator <path> --optimizer <path-to-wasm-opt.exe>
WasmBridge.exe generate-fallback --wasm module.wasm --out candidate.js --validator <path> --wasm2js <path> --esbuild <path> [--global WasmBridgeGeneratedCandidate]
Safety gate: Never replace buffers-fallback.js merely because generation succeeded. First run Examples/BufferArena/generated-fallback-probe.html in Firefox 52.9 ESR on XP. The candidate must expose the expected allocator/memory ABI and produce byte-identical results against the known-good fallback.
```

9. Current repository map

```
Path | Purpose
WasmBridge.sln | Traditional VS solution with managed API, CLI, and native DLL projects.
```

Compiler/WasmCompiler.cs | Minimal freestanding wasm32 Clang wrapper; multi-export support and exported memory.
Compiler/ExternalToolchain.cs | WABT/Binaryen/esbuild external tool wrappers.
Compiler/WasmPackage.cs | Bundle creation plus SHA-256 manifest; optional adapter.js.
CLI/Program.cs | Commands: build, package, verify, self-test, validate-legacy, optimize-legacy, generate-fallback.
Core/buffers.c/.h | Portable bounded allocator and RGBA processing ABI.
Runtime/wasmbridge.js | Backend selection, export validation, optional XHR byte loader support.
Runtime/buffers.js | General buffer handles, typed read/write copies, memory growth, bounds checks.
Runtime/legacy-compat.js | Optional FF52-era capability helper and XHR loader.
Examples/BufferArena/index.html | Primary dynamic-memory browser demo and comparative benchmark.
Examples/BufferArena/generated-fallback-probe.html | Acceptance gate for wasm2js-generated fallback.
Documentation/FF52_COMPATIBILITY.md | Detailed legacy-browser contracCompilerack.

{TCT /r.8 T .m.c8.8 T/r.8 T .m.csTCT490k.

Run existing regressions: Run Node tests and portable native tests available in Tests/. Preserve current browser examples as regression fixtures.
Exercise WABT/Binaryen/esbuild wrappers: Use real installed tools on the build host. Validate buffers.wasm under the legacy feature gate, optimize a copy, generate the JS candidate, and inspect outputs. Do not overwrite working assets.
Firefox 52 generated-fallback acceptance: Open generated-fallback-probe.html on XP Firefox 52.9 ESR. Fix adapter/normalization issues until the candidate matches the handwritten fallback. Only then consider making auto-generated fallback a supported path.
Implement general module API: Consolidate loader and memory operations into a reusable API while keeping the proven low-level code available. Add tests for multiple modules, imports, direct calls, buffers, strings and typed numeric arrays.
Add Emscripten backend later: After the small freestanding path is solid, investigate an Emscripten compiler backend for larger C/C++ libraries. Keep it optional and build-host only; verify actual generated output against Firefox 52 rather than trusting nominal browser-target flags.

12. Acceptance criteria for the next milestone

* Release x64 and x86 Windows XP targets build from WasmBridge.sln without AIEXE/AI_RUN dependencies.
* Direct x64 self-test from the build output loads WasmBridge.Native.dll and completes all existing managed/native checks with exit code 0.
* Existing HelloWorld, ImageProcessing, and BufferArena pages still pass in Firefox 52.9 ESR on XP.
* No shipped legacy runtime JS introduces unsupported syntax or an ES-module dependency.
* WABT legacy validation and Binaryen/esbuild generation run successfully on the build host with explicitly recorded tool versions.
* Generated fallback candidate passes the dedicated FF52 probe before it is used as a default fallback.
* New general API has regression tests for multiple module instances, function invocation, allocation/release, typed buffers and fallback selection.
* No public API retains stale views into Wasm memory after memory.grow.
* Diagnostics clearly distinguish compile-time validation, browser instantiation failure, ABI validation failure and deliberate JS fallback.

13. Known limitations and guardrails

* BufferArena is a bounded prototype: 128 tracked blocks, 32 MiB adapter/allocator budget, 16 MiB maximum individual JS allocation, first-fit reuse, no coalescing or shrinking.
* The raw C ABI cannot perfectly detect every stale pointer after address reuse; the JS handle layer provides stronger use-after-release protection.
* Current browser timing numbers include allocation and memory copies. Do not market them as a general Wasm-vs-JS benchmark.
* Package verification currently includes header checks plus optional WABT validation. It is not a security sandbox or proof of browser compatibility.
* Generated wasm2js output may need a module-specific adapter. Syntax transpilation does not guarantee identical memory/import semantics.
* No upstream compiler/optimizer source or binaries are currently vendored by the integration. If source is incorporated later, retain and review the relevant third-party licenses/notices. Emscripten is permissively licensed (MIT/NCSA options for its main code); Binaryen and WABT use Apache-2.0 for their projects. Verify exact files/dependencies before redistribution.

14. Recommended first instruction to CODEX

```
Start here: Open TheKernelDoc/WasmBridge and treat the repository as fully under your control. Do not use
AI_RUN, AIEXE, DBGRun orchestration, or the old mirrored R: workflow. First establish a clean direct Release
x64/x86 build from WasmBridge.sln, run the existing tests from that worktree, and resolve the direct
WasmBridge.Native.dll self-test. Preserve the verified Firefox 52.9 ESR behavior. Then exercise the
WABT/Binaryen/esbuild integration and make the generated fallback pass generated-fallback-probe.html on
Firefox 52.9 ESR before changing any default fallback. After that, begin consolidating the loader and
BufferArena into a general multi-module WasmBridge API while retaining XP/.NET 4.0/v141_xp compatibility.
```

15. Handoff provenance

Repository source used for this document: Magneticon/WasmBridge, main commit 8a3496e763b5fa279e465d4292c5388fb05d1c7c ("Record XP BufferArena success and document optional FF52 toolchain and no-silent-replacement policy"). The target repository supplied by the user is https://github.com/TheKernelDoc/WasmBridge.

The document also incorporates the latest user-observed test results from this project session, including the successful XP Firefox BufferArena run and the native DLL path/dependency investigation. Where repository documentation lagged the latest screenshots, this handoff records the newer observed state explicitly.

End of handoff